

REDES NEURAIS ARTIFICIAIS E MACHINE LEARNING

TEMA 1 – ALGORITMOS DE AGLOMERAÇÃO (CLUSTERING E DBSCAN)

Os algoritmos de aglomeração, denominados sob o termo “*clustering*”, referem-se a um tipo de algoritmo genérico usado para designar um amplo espectro de métodos numéricos de análise de dados com o objetivo de descobrir grupos homogêneos de objetos. A tarefa de agrupamento de objetos em diferentes grupos pode permitir a representação de maneira conveniente de bases de dados muito grandes, visando facilitar a compreensão. Estes algoritmos são caracterizados pela aprendizagem de maneira não supervisionada (Duda; Hart; Stork, 2001, p. 13).

A análise de grupos pode ser definida como a organização de um conjunto de objetos em grupos baseados na similaridade ou diferença entre eles. Pode-se afirmar que é o processo de particionar um conjunto em subconjuntos, de forma que os objetos nestes conjuntos compartilhem características comuns e sejam significativamente diferentes entre os subconjuntos.

Para ilustrar isso, dois indicadores podem auxiliar a descrever o desempenho de uma análise de grupos (Castro; Ferrari, 2016):

- **Coesão interna** ou **homogeneidade**: permite demonstrar o quanto os objetos que pertencem a um grupo compartilham das mesmas características.
- **Isolamento externo**: permite verificar o quanto os elementos de um grupo são separados dos elementos de outro grupo.

Os dados são alimentados aos algoritmos na forma de um **vetor de características**: um vetor que contém os atributos relativos aos objetos que estão sendo analisados. São chamados também de **dimensões**. Os atributos variam em uma faixa de valores, conforme as suas propriedades dentro de um domínio sendo abordado. Por exemplo, idade, altura, largura, quantidade vendida etc. Dessa forma, um objeto tem um conjunto de valores específicos para cada atributo que o define.

Na Figura 1, pode-se identificar o vetor de características para o dataset popular Iris¹. Este banco de dados refere-se a um conjunto de 150 exemplos de flores Iris, contendo quatro atributos: comprimento e largura da pétala, comprimento e largura da sépala. Desta forma, o vetor de características contém

¹ <https://archive.ics.uci.edu/ml/datasets/iris>.

quatro atributos, e cada instância dentro do dataset contém valores para cada atributo. Cada instância pode ser utilizada para efetuar o treinamento do algoritmo de aglomeração e permitir identificar grupos de flores com características comuns.

Figura 1 - Exemplos de vetor de características do conjunto de dados Iris

	sepal-length	sepal-width	petal-length	petal-width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Fonte: UCI.

Algoritmos de aglomeração são utilizados em uma variedade de áreas:

- **Marketing:** segmentação de clientes e de produtos
- **Agricultura:** categorização de plantas, solos e frutos
- **Meteorologia:** identificação de padrões climáticos
- **Finanças:** identificação de perfil de clientes
- **Medicina:** categorização de pacientes, diagnósticos e remédios

Deve-se ter em mente a diferença entre agrupamento e classificação. Na classificação, tal como foi estudado anteriormente com o perceptron, a base de dados é previamente rotulada, ou seja, sabe-se a qual classe se refere uma instância dentro do dataset. No algoritmo de classificação, busca-se identificar a qual classe uma nova instância, externa ao dataset, pode pertencer. Já na tarefa de agrupamento, não existe um conhecimento a priori sobre classes ou rótulos a que pertencem as instâncias. Na verdade, o processo de agrupamento é utilizado justamente para a definição de grupos existentes dentro do dataset.

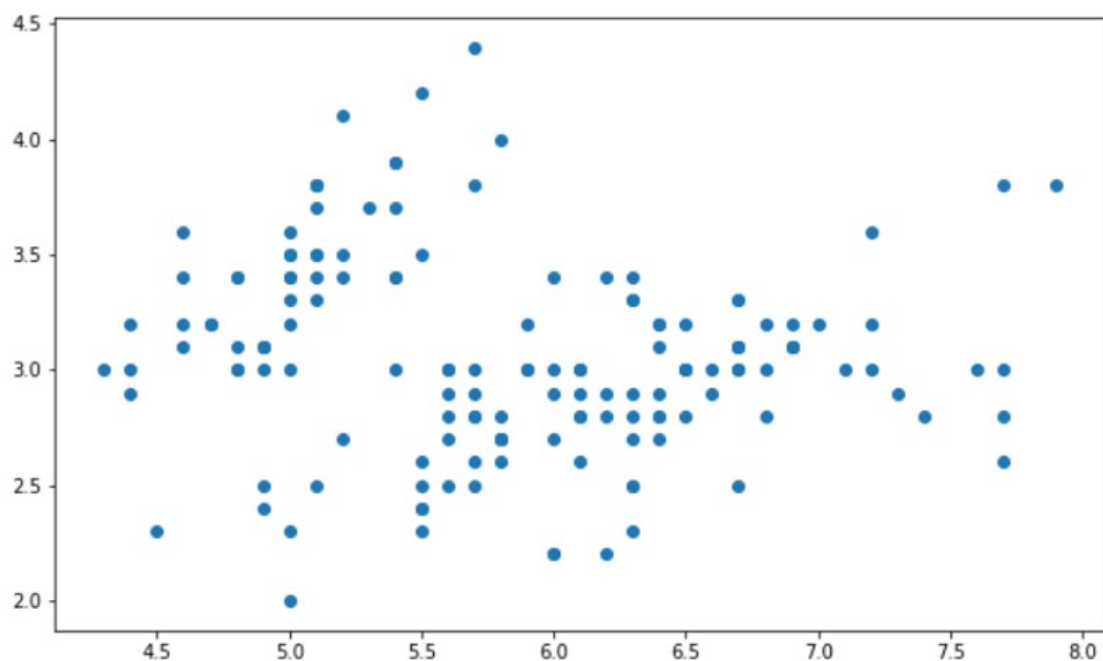
Existem vários algoritmos para agrupamento, dentre os quais pode-se citar (Castro; Ferrari, 2016):

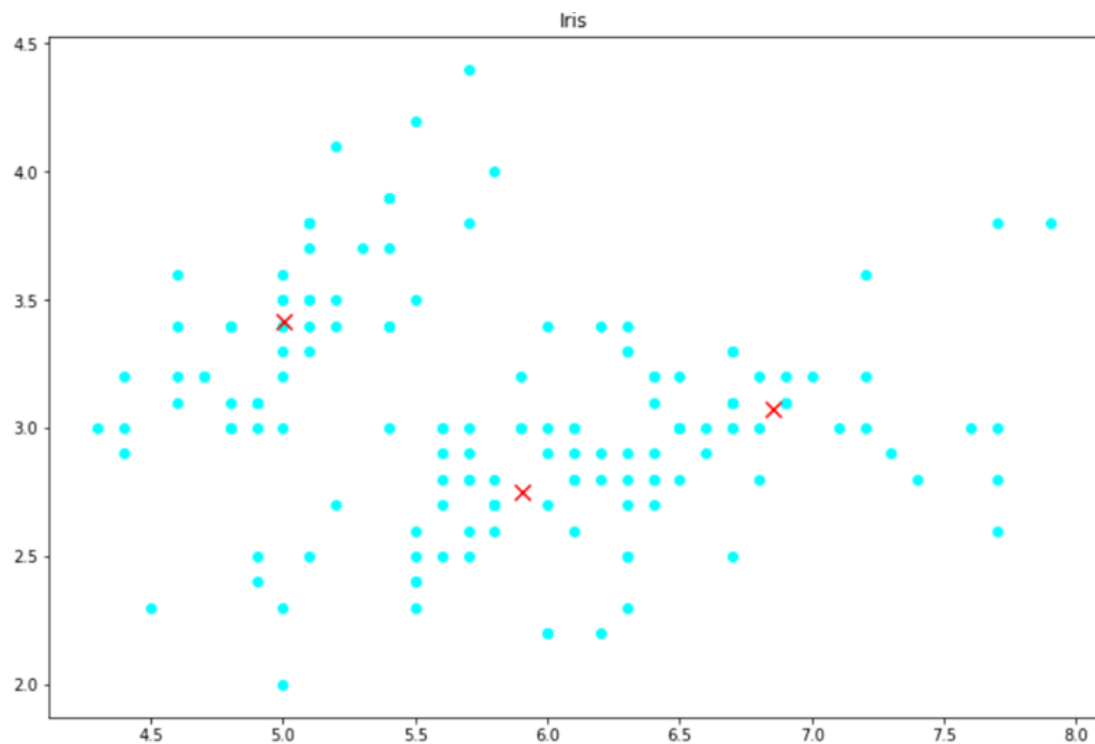
- Baseado em protótipo: *K-means* e *k-medoids* (Amaral, 2016, p. 109)
- Baseado em densidade: DBSCAN (Amaral, 2016, p. 110)
- *Clustering* Hierárquico (*Single linkage*) (Amaral, 2016, p. 111)

- Árvore geradora mínima (MST – *minimum spanning tree*) (Castro; Ferrari, 2016, p. 127)
- *Fuzzy k-means* (Duda; Hart; Stork, 2001, p. 14)

Na Figura 2, ilustra-se o exemplo de aplicação do algoritmo de aglomeração k-means para o conjunto de dados Iris. No eixo horizontal é colocado o comprimento da sépala e no eixo vertical a largura da sépala. As 150 instâncias são, portanto, pontos sobre o gráfico. O algoritmo k-means é treinado para identificar três grupos (*clusters*) conforme o posicionamento destes pontos, sendo mostrado no gráfico a seguir. Cada um dos três *clusters* é representativo do conjunto de instâncias que estão próximas a eles.

Figura 2 - Exemplo de aplicação do algoritmo de aglomeração k-means sobre o dataset Iris





O algoritmo k-means é uma alternativa que permite obter rapidamente um conjunto de centros comuns para agrupamentos. A ideia é que um conjunto de vetores comuns ou *clusters* seja representativo dos grupos existentes em um conjunto de dados (Medeiros, 2007).

O algoritmo DBSCAN, além da distribuição de instâncias, considera também a densidade, com a concentração das instâncias influenciando na determinação dos centros comuns (*clusters*).

TEMA 2 – CONSTRUÇÃO DO MODELO EM PYTHON E JUPYTER NOTEBOOK (IRIS)

Um dos *datasets* mais estudados em Machine Learning é o Iris. Este banco de dados foi publicado por Fischer em 1936, tornando-se popular na literatura de reconhecimento de padrões. Como visto anteriormente, Iris contém 150 instâncias com quatro atributos diferentes. Este dataset é pré-classificado, dividindo as instâncias em três rótulos diferentes: Setosa, Versicolour e Virginica. Esta pré-classificação, no entanto, em atividades de análise de agrupamento, pode ser inicialmente ignorada e permite que seja feito posteriormente o teste da qualidade do agrupamento obtido.

A biblioteca Scikit-learn é adequada para abordar uma solução para o problema dentro da linguagem Python. Ela inclui vários algoritmos de

agrupamento e é projetada para trabalhar com as bibliotecas NumPy e SciPy. Ela permite também lidar com os dados dentro das estruturas de dados oferecidas pela biblioteca Pandas. Para o problema aqui proposto, as bibliotecas Pandas e Matplotlib serão importadas, junto com o módulo do k-means, que é parte da Scikit-learn (Figura 3).

Figura 3 - Descrição das bibliotecas a serem importadas para a análise de agrupamentos do dataset Iris

```
In [1]: 1 import pandas as pd
        2 import matplotlib.pyplot as plt

In [2]: 1 from sklearn.cluster import KMeans
```

O script para a análise de agrupamentos com o dataset Iris começa com a leitura do próprio dataset, considerando-se os atributos e excluindo-se inicialmente o atributo do rótulo de cada instância. Esta leitura pode ser feita no próprio site do repositório em que se encontra o dataset, ou pode ser lido a partir dos dados instalados localmente. Na Figura 4 está a parte do script que executa a leitura dos dados a partir do repositório diretamente. É utilizado o comando `read_csv()` da biblioteca Pandas para colocar os dados em um DataFrame, pois o arquivo está em formato CSV originalmente, não utilizando-se cabeçalho (`header=None`). As colunas serão colocadas posteriormente com o atributo `columns` do DataFrame. Logo em seguida, o comando `head()` é chamado para fazer-se a impressão parcial do conjunto de dados que foram lidos, contendo assim os quatro atributos.

Figura 4 - Script com a leitura do conjunto de dados Iris

```
In [2]: data = pd.read_csv("https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data", header=None)
        data.columns = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'class']
        del data['class']
        data.head()
```

```
Out[2]:
```

	sepal-length	sepal-width	petal-length	petal-width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Sempre é importante ter uma visão geral sobre como os dados de um dataset se apresentam. Como feito anteriormente no estudo do MLP, o comando para exibir a estatística descritiva dos dados existente no Pandas, `describe()`, é executado e um resumo dos dados é mostrado. A estatística descritiva mostra a média de cada atributo, a mediana (50%), o desvio-padrão, os valores mínimos e máximos de cada um dos atributos, e a informação sobre o primeiro (25%) e o terceiro (75%) quartis. Os dados são, de certa forma, bastante homogêneos, visto se tratar de uma medida de tamanho em centímetros.

Figura 5 - Estatística descritiva das instâncias do dataset Iris

```
In [4]: 1 # Fazendo a estatística descritiva
        2 data.describe()
```

Out[4]:

	sepal-length	sepal-width	petal-length	petal-width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Os quatro atributos podem ser relacionados entre si dois a dois para gerar gráficos de duas dimensões para a visualização dos dados. Com a biblioteca Matplotlib, pode-se construir os gráficos bidimensionais utilizando-se dois atributos e visualizando como eles se distribuem. Na Figura 6 encontra-se um script para executar a visualização.

Figura 6 - Script para visualização dos dados em gráficos bidimensionais

```
In [5]: 1 # Gerando o gráfico, mostrando apenas o relacionamento entre o comprimento e largura da sépala
        2 fig = plt.figure(figsize=(10,6))
        3 sp = fig.add_subplot(1,1,1)
        4 sp.scatter(data['sepal-length'],data['sepal-width'])
```

A construção e impressão do gráfico com a Matplotlib necessita de três etapas: a configuração da área do gráfico com `figure()`, a criação de um objeto para adicionar um gráfico especificamente (no caso, com o comando `add_subplot()`; uma área pode conter mais de um gráfico), e a impressão

propriamente dita, em que será utilizado o tipo de gráfico *scatter()*, no qual os dois atributos são informados para a criação do gráfico e cada instância será então um ponto plotado na área do gráfico. Diferentes gráficos podem ser feitos com o uso de diferentes combinações dos atributos entre si.

TEMA 3 – PRÁTICA DO MODELO (IRIS)

Com o modelo da análise de agrupamento construído, parte-se para a sua execução propriamente dita. Pode-se notar que esta parte é similar à fase de treinamento do MLP estudado anteriormente. A ideia agora é gerar um objeto de análise de agrupamentos com o algoritmo k-means. Será então criado um objeto *KMeans()* com alguns parâmetros que irão caracterizar inicialmente os centros comuns do modelo e que serão refinados a cada execução do algoritmo. Alguns parâmetros são:

- **n_clusters:** número de *clusters*;
- **init:** algoritmo que será executado para a criação dos centros comuns (pode ter dois tipos, 'kmeans++', que permite acelerar a convergência, e 'random', que irá escolher instâncias aleatoriamente para os centros comuns);
- **n_init:** número de vezes que o algoritmo irá iniciar aleatoriamente para pegar a melhor configuração de *clusters* iniciais;
- **max_iter:** número máximo de iterações que o algoritmo será executado;
- **tol:** tolerância entre duas iterações para estabelecer como critério de parada do algoritmo;
- **random_state:** número aleatório utilizado como semente para o algoritmo de geração randômica dos centros.

Na Figura 7 consta o script para a criação do objeto *KMeans()* na análise de agrupamento do conjunto Iris. A execução do comando é simples e cria o objeto pelo qual os *clusters* podem ser identificados para cada instância dentro do conjunto. Pode-se notar os parâmetros para a criação do objeto: serão três *clusters*, o algoritmo de inicialização dos centros comuns é o 'k-means++', o número de execuções aleatórias para pegar a melhor configuração inicial será de 10, e um número aleatório qualquer é colocado como semente para o gerador randômico para os centros.

Figura 7 - Execução do script KMeans()

```
In [5]: kmeans = KMeans(n_clusters=3, init='k-means++', n_init=10, random_state=1234)
kmeans

Out[5]: KMeans(n_clusters=3, random_state=1234)
```

Após a criação do objeto KMeans(), já é possível prever as classes a que pertencem cada uma das instâncias. Utiliza-se o comando *fit_predict()* do próprio objeto, de maneira a criar um novo atributo para o DataFrame denominado de “clusters”. Na Figura 8 é mostrado o script e o resultado da execução, sendo mostradas algumas instâncias.

Figura 8 - Execução do script para predição das instâncias, baseados no algoritmo de agrupamento

```
In [6]: data["clusters"] = kmeans.fit_predict(data)
data

Out[6]:
```

	sepal-length	sepal-width	petal-length	petal-width	clusters
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	1
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	1

150 rows × 5 columns

Pode-se verificar a nova coluna aparecendo à direita nas instâncias mostradas (numerada com 0, 1 e 2). Estas classes preditas são obtidas apenas considerando-se os dados para a análise de agrupamento, podendo ser comparadas com as classes prévias para verificar o quanto a análise se aproxima do modelo real de classificação.

Para saber a quantidade de instâncias analisadas, pode-se utilizar um script com *groupby()* da biblioteca Pandas. Este comando (de forma similar à cláusula GROUP BY em comandos SQL para bancos de dados) agrupa as instâncias e calcula a quantidade de cada classe atribuída. Na Figura 9, verifica-se a quantidade de cada um. Note que o processo é estocástico, no sentido da

inicialização aleatória do algoritmo k-means. Portanto, diferentes parâmetros e execuções poderão gerar diferentes distribuições de instâncias pelas classes. Note também a chamada ao atributo `size()`, que faz a totalização e a inclusão da coluna das contagens (“counts”).

Figura 9 - Sumário do número de instâncias a partir da execução do algoritmo k-means

```
In [7]: group_count = data.groupby("clusters").size().reset_index(name='counts')
group_count
```

Out[7]:

	clusters	counts
0	0	50
1	1	62
2	2	38

Os centros comuns às classes ou *clusters* conterão também os quatro atributos, da mesma forma que as instâncias. A análise com o algoritmo k-means permite a obtenção dos valores destes *clusters* para cada atributo. Para saber estes valores, um script pode ser gerado novamente com o comando `groupby()`. Após este comando, calcula-se os valores dos centros comuns, que nada mais serão do que as médias, a partir da execução de `mean()`. Na Figura 10, está mostrada a execução deste script, com os valores dos atributos para cada centro comum.

Figura 10 - Cálculo dos valores dos *clusters* para a análise de agrupamento do conjunto Iris

```
In [8]: group = data.groupby("clusters")
group
```

Out[8]: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x0000027A2DDC6FD0>

```
In [9]: gm = group.mean()
gm
```

Out[9]:

	sepal-length	sepal-width	petal-length	petal-width
clusters				
0	5.006000	3.418000	1.464000	0.244000
1	5.901613	2.748387	4.393548	1.433871
2	6.850000	3.073684	5.742105	2.071053

Além da visualização quantitativa, pode-se construir um gráfico para mostrar a distribuição das instâncias e onde está posicionado os centros comuns para cada rótulo. O próximo script na Figura 11 mostra o uso de comandos da biblioteca Matplotlib para a elaboração deste gráfico. O comando `figure()` do objeto `plt` configura a área de impressão e o comando `add_subplot()` adiciona uma subárea. Em seguida, com o comando `scatter()` os pontos relativos às instâncias são plotados com uma cor específica. Usa-se o mesmo comando `scatter()` para imprimir os centros comuns com os marcadores em “x”. Ao final, é colocado o título do gráfico. Na parte inferior da Figura 2 consta o gráfico que foi gerado.

Figura 11 - Script para impressão em gráfico dos centros comuns do conjunto Iris.

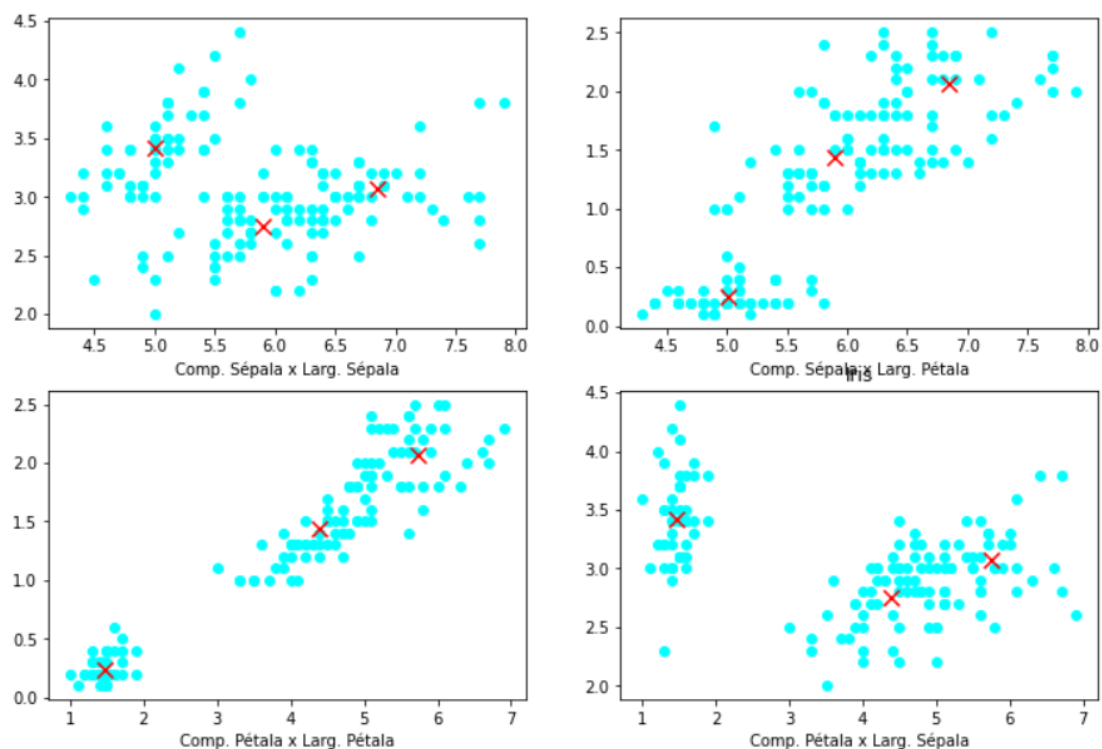
```
In [10]: fig = plt.figure(figsize=(12,8))
         sp = fig.add_subplot(1,1,1)
         sp.scatter(data['sepal-length'],data['sepal-width'], color='cyan')
         sp.scatter(gm['sepal-length'],gm['sepal-width'], marker='x', s=100, color='red')
         plt.title("Iris")
```

Pode-se criar um script mais completo, com todas as possibilidades de cruzamento dos atributos. Para isso, utiliza-se uma área subdividida em quatro regiões, cada uma referenciada pelo comando `add_subplot()` específico. Os atributos são modificados para cada um, bem como um subtítulo para indicar a diferenciação entre eles. Na Figura 12 está o script completo, com o gráfico resultante na Figura 13.

Figura 12 - Script para construção dos quatro gráficos para a análise de agrupamento do conjunto Iris

```
In [23]: fig = plt.figure(figsize=(12,8))
sp = fig.add_subplot(2,2,1)
sp.scatter(data['sepal-length'],data['sepal-width'], color='cyan')
sp.scatter(gm['sepal-length'],gm['sepal-width'], marker='x', s=100, color='red')
sp.set_xlabel("Comp. Sépala x Larg. Sépala")
sp = fig.add_subplot(2,2,2)
sp.scatter(data['sepal-length'],data['petal-width'], color='cyan')
sp.scatter(gm['sepal-length'],gm['petal-width'], marker='x', s=100, color='red')
sp.set_xlabel("Comp. Sépala x Larg. Pétala")
sp = fig.add_subplot(2,2,3)
sp.scatter(data['petal-length'],data['petal-width'], color='cyan')
sp.scatter(gm['petal-length'],gm['petal-width'], marker='x', s=100, color='red')
sp.set_xlabel("Comp. Pétala x Larg. Pétala")
sp = fig.add_subplot(2,2,4)
sp.scatter(data['petal-length'],data['sepal-width'], color='cyan')
sp.scatter(gm['petal-length'],gm['sepal-width'], marker='x', s=100, color='red')
sp.set_xlabel("Comp. Pétala x Larg. Sépala")
plt.title("Iris")
```

Figura 13 - Gráficos das relações entre os atributos das instâncias e a posição dos centros comuns de cada classe



TEMA 4 – CONSTRUÇÃO DO MODELO EM PYTHON E JUPYTER NOTEBOOK (WHOLESALE CUSTOMER)

O próximo exemplo de análise de agrupamento será feito com o dataset denominado “Wholesale Customer” (cliente de atacado). Este dataset² contém

² <https://archive.ics.uci.edu/ml/datasets/wholesale+customers>

440 instâncias, que são clientes, com o valor total de vendas anuais em cada um dos seis atributos: alimentos frescos (“fresh”), derivados de leite (“milk”), gêneros alimentícios (“grocery”), congelados (“frozen”), higiene e limpeza (“detergents & paper”) e doces (“delicatessen”) (Abreu, 2011).

O objetivo da análise de agrupamento será o de identificar grupos de clientes com perfis de compra similares quanto aos valores comprados. Para facilitar a análise, dois campos serão excluídos neste momento, canal e região. O script da Figura 14 mostra a leitura do arquivo diretamente do repositório, juntamente com os comandos para exclusão dos campos que não serão utilizados, e o comando *head()* para mostrar as primeiras cinco instâncias do dataset.

Figura 14 - Script para leitura do dataset Wholesale Customer e visualização das primeiras instâncias

```
In [21]: data = pd.read_csv("https://archive.ics.uci.edu/ml/machine-learning-databases/00292/wholesale%20customers%20data.csv")
del data["Channel"]
del data["Region"]
data.head()
```

Out[21]:

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
0	12669	9656	7561	214	2674	1338
1	7057	9810	9568	1762	3293	1776
2	6353	8808	7684	2405	3516	7844
3	13265	1196	4221	6404	507	1788
4	22615	5410	7198	3915	1777	5185

A estatística descritiva com *describe()* mostra um resumo dos dados para cada grupo alimentício, com relação às 440 instâncias. Além da contagem, aparecem também a média do valor comprado, o desvio-padrão, os valores mínimos e máximos, a mediana (50%) e o primeiro e terceiro quartil (25% e 75%, respectivamente). Com relação aos valores mínimos e máximos, pode-se notar uma faixa muito ampla de variação, o que pode ser identificado pela comparação do desvio-padrão com os valores médios (Figura 15).

Os próximos scripts farão a preparação para a execução do algoritmo k-means. A análise de agrupamento será feita com cinco *clusters*, com a inicialização aleatória dos centros começando dez vezes, o algoritmo utilizado será o “k-means++”. O script na Figura 16 mostra a preparação do algoritmo. A colocação em outro DataFrame visualizada no primeiro comando é opcional.

Figura 15 - Estatística descritiva dos dados do dataset Wholesale customer

```
In [22]: # Fazendo-se a estatística descritiva
data.describe()
```

Out[22]:

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
count	440.000000	440.000000	440.000000	440.000000	440.000000	440.000000
mean	12000.297727	5796.265909	7951.277273	3071.931818	2881.493182	1524.870455
std	12647.328865	7380.377175	9503.162829	4854.673333	4767.854448	2820.105937
min	3.000000	55.000000	3.000000	25.000000	3.000000	3.000000
25%	3127.750000	1533.000000	2153.000000	742.250000	256.750000	408.250000
50%	8504.000000	3627.000000	4755.500000	1526.000000	816.500000	965.500000
75%	16933.750000	7190.250000	10655.750000	3554.250000	3922.000000	1820.250000
max	112151.000000	73498.000000	92780.000000	60869.000000	40827.000000	47943.000000

Figura 16 - Script para preparação dos dados e do algoritmo k-means

```
In [23]: # Atribuindo-se os valores a outro data frame
data_array = data.values
```

```
In [24]: # Cria-se o objeto KMeans com 5 clusters
kmeans = KMeans(n_clusters=5, init='k-means++', n_init=10, random_state=1234)
kmeans
```

Out[24]: KMeans(algorithm='auto', copy_x=True, init='k-means++', max_iter=300, n_clusters=5, n_init=10, n_jobs=1, precompute_distances='auto', random_state=1234, tol=0.0001, verbose=0)

Com seis atributos, a análise dos dados mediante gráficos como feito no exemplo do conjunto Iris fica um pouco mais complexa, pois seriam necessários 15 gráficos para relacionar 2 a 2 todos os atributos. No entanto, é possível fazer a impressão dos gráficos de maneira a indicar os centros comuns por grupo na forma de barras para cada atributo, tornando possível a visualização dos perfis de compra de cada grupo.

TEMA 5 – PRÁTICA DO MODELO (WHOLE SALE CUSTOMER)

A execução do algoritmo de agrupamento para o banco de dados Wholesale customer é feita utilizando-se o método *fit_predict()* do objeto KMeans() criado. Na Figura 17 é mostrado o script e a impressão das instâncias com os atributos e uma nova coluna relativa ao *cluster* ao qual a instância pertence, sendo os grupos numerados de 0 a 4. A visualização dos dados pode ser melhorada se a listagem for colocada em ordem de *clusters*. Com a biblioteca Pandas, é possível utilizando o método *sort_values()* do objeto DataFrame (Figura 18).

Figura 17 - Script para execução do algoritmo de agrupamento e a visualização das instâncias com a nova coluna relativa aos centros comuns de cada grupo

```
In [25]: # Gera-se uma nova coluna no data frame
data["clusters"] = kmeans.fit_predict(data_array)
data
```

```
Out[25]:
```

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen	clusters
0	12669	9656	7561	214	2674	1338	0
1	7057	9810	9568	1762	3293	1776	0
2	6353	8808	7684	2405	3516	7844	0
3	13265	1196	4221	6404	507	1788	0
4	22615	5410	7198	3915	1777	5185	3
5	9413	8259	5126	666	1795	1451	0
6	12126	3199	6975	480	3140	545	0
7	7579	4956	9426	1669	3321	2566	0
8	5963	3648	6192	425	1716	750	0
9	6006	11093	18881	1159	7425	2098	1
10	3366	5403	12974	4400	5977	1744	1
11	13146	1124	4523	1420	549	497	0
12	31714	12319	11757	287	3881	2931	3
13	21217	6208	14982	3095	6707	602	3

Figura 18 - Ordenação dos dados por *clusters*

```
In [26]: # Ordenando-se pelos clusters
data.sort_values(by="clusters")
```

```
Out[26]:
```

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen	clusters
0	12669	9656	7561	214	2674	1338	0
252	6623	1860	4740	7683	205	1693	0
250	3191	1993	1799	1730	234	710	0
249	8040	3795	2070	6340	918	291	0
247	13569	346	489	2077	44	659	0
246	8885	2428	1777	1777	430	610	0
244	6202	7775	10817	1183	3143	1970	0
243	11210	3576	5119	561	1682	2398	0
238	7363	475	585	1112	72	216	0
236	8635	820	3047	2312	415	225	0
235	1838	6380	2824	1218	1216	295	0
233	964	4984	3316	937	409	7	0

Da mesma forma que efetuado para o conjunto Iris, para saber a quantidade de instâncias analisadas, pode-se utilizar um script com `groupby()` da biblioteca Pandas. Na Figura 19 é possível verificar a quantidade de instâncias para cada um. Como afirmado anteriormente, diferentes parâmetros e execuções poderão gerar diferentes distribuições de instâncias pelas classes. Note também a chamada ao atributo `size()`, que faz a contagem e a inclusão da coluna das contagens (“counts”) no resultado do agrupamento. Pode-se notar a maior quantidade (226) para a classe ‘0’ e a menor (7) para a classe ‘4’.

Figura 19 - Contagem de instâncias por *clusters*

```
In [27]: # Utilizando-se GroupBy do Pandas
group = data.groupby("clusters")

In [28]: # Contabilizando-se itens por cluster
group_count = data.groupby("clusters").size().reset_index(name='counts')
group_count
```

Out[28]:

	clusters	counts
0	0	226
1	1	81
2	2	21
3	3	105
4	4	7

O cálculo dos centros comuns pode ser feito também com a execução do comando `mean()`. Na Figura 20 podem ser visualizados os valores médios de cada *cluster*, para cada atributo.

Figura 20 - Visualização dos valores médios dos *clusters* para os atributos considerados

```
In [29]: # Obtendo-se os valores médios (centros) para cada cluster
group.mean()
```

Out[29]:

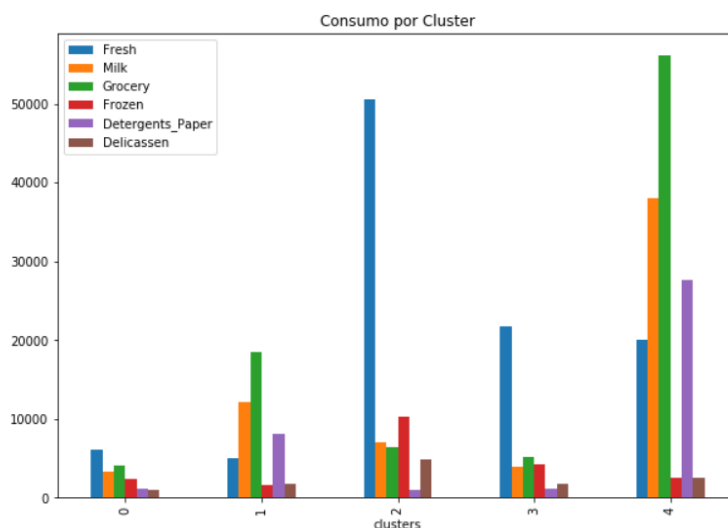
	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
clusters						
0	6143.871681	3276.792035	4115.163717	2438.349558	1220.703540	992.539823
1	4991.481481	12140.543210	18505.506173	1591.111111	8148.246914	1822.407407
2	50512.095238	6987.523810	6478.095238	10215.380952	1030.523810	4904.761905
3	21774.600000	3934.209524	5149.200000	4183.123810	1112.542857	1696.923810
4	20031.285714	38084.000000	56126.142857	2564.571429	27644.571429	2548.142857

Para a visualização na forma gráfica, utiliza-se o próprio objeto “group” criado e o gráfico é construído a partir do próprio cálculo de médias. Para isso, usa-se o método `plot()`, com o parâmetro `kind='bar'` (gráfico do tipo barras), o qual gera o valor de compra de cada atributo, para cada *cluster*. O que se observa na impressão do gráfico é a caracterização dos grupos tendo perfis definidos quanto à compra de cada atributo. O grupo ‘0’ é o grupo que tem baixo volume de compra de maneira geral em todos os atributos (lembrando que é o maior em número de clientes, como visto anteriormente). O grupo ‘1’ caracteriza-se por um valor ligeiramente maior de gêneros alimentícios e derivados de leite. O grupo ‘2’ é característico pelo fato de a compra de alimentos frescos ser muito maior do que os outros atributos, sendo os congelados em segundo lugar. O grupo ‘3’ também tem esta mesma característica, porém um pouco menor quanto ao atributo de alimentos frescos. E o grupo ‘4’ é o que contém um alto nível de compra de gêneros alimentícios, seguido de derivados de leite e de higiene e limpeza, bem como alimentos frescos; entretanto, deve ser lembrado que este grupo contém o menor número de clientes (7).

Figura 21 - Gráfico com a visualização dos *clusters* na forma de grupos de barras para cada atributo

```
In [30]: # Gerando-se o gráfico, mostrando as classes como conjuntos de barras, cada uma representando uma característica
group.mean().plot(kind="bar", figsize=(10,7), title="Consumo por Cluster")

Out[30]: <matplotlib.axes._subplots.AxesSubplot at 0x14247362908>
```



Este exemplo de análise de agrupamentos orientada pelos dados mostra o potencial de aplicação deste tipo de algoritmo para a compreensão de dados sobre os quais não se tem algum conhecimento prévio.

REFERÊNCIAS

ABREU, N. G. C. F. M. de. **Análise do perfil do cliente Recheio e desenvolvimento de um sistema promocional**. 2011. - ISCTE-IUL, Lisboa, [S. l.], 2011. Disponível em: <<http://hdl.handle.net/10071/4097>>. Acesso em: 4 ago. 2021.

AMARAL, F. **Introdução à Ciência de Dados: mineração de dados e Big Data**. Rio de Janeiro: Alta Books, 2016.

CASTRO, L. N. de; FERRARI, D. G. **Introdução à Mineração de Dados: conceitos básicos, algoritmos e aplicações**. São Paulo: Editora Saraiva, 2016.

DUDA, R. O.; HART, P. E.; STORK, D. G. **Pattern Classification**. 2. ed. New York: Wiley, 2001.

MEDEIROS, L. F. **Redes neurais em Delphi**. 2. ed. Florianópolis: Visualbooks, 2007.

UCI. Centro de aprendizado de máquina e sistemas inteligentes. **Conjunto de dados Iris**. [S.d.]. Disponível em: <<https://archive.ics.uci.edu/ml/datasets/iris>>. Acesso em: 04 ago. 2021.